

이번 학습을 통해 "게임을 만드는 법" 자체가 주제가 아니라, AI를 이용해서 소프트웨어와 하드웨어를 점점 연결해 가는 과정을 배우는 시간을 가졌다.

우선 초반이해를 위해 전문을 가볍게 훑어 보았다.

초반 게임설계과정에서 제미나이를 통해 테스트벤치를 만들어 동작도 확인하며 추가적인 기능도 생성, 그러다보니 단순한 디지털파형으로만 구성되어있어 그래픽을 붙이고 이후에도 각종 오류나 기능상 아쉬운점을 개선해나간다. 이후 소프트웨어로 구현된게임의 일부기능을 하드웨어로 구현하면 어떨까하는 궁금증에서 이걸 단순히 게임을만드는 법이아닌, 소프트웨어와 하드웨어의 연결과정, 즉 교수님께서 만들어놓은 예제 프로젝트를 실행하며 그 구조와 하드웨어 설계과정을 이해하는 실습이란걸 깨달았다.

예제 프로젝트를 순차적으로 실행하며 따라가면 배운것들 일부를 정리해본다.

(Verilog 하드웨어가 clk에 맞춰 동작하면서 x_pos, y_pos 같은 위치 신호를 계속 변화시키고, 그 신호를 이용해 화면의 픽셀을 만들어낸다.)

(Q01~Q02에서 우리가 배운 것

소프트웨어 관점

게임 프로그램 실행 → 화면 출력

하드웨어 관점

클럭 → 여러 모듈 동시 동작 → 위치/상태 신호 변화 → 픽셀 신호 → 화면)

(Q06에서 볼 핵심 교수님이 "포탄을 추가한다"는 건 단순히 총알 이미지 하나 추가가 아님. 하드웨어 입장에서

총알 위치 → 이동 → 충돌 판정 → 화면 픽셀 → 게임 상태 변화

이런 것들을 회로로 만들어야 한다는 뜻이다.

이게 바로 이 글이 단순 게임 제작 글이 아니라 하드웨어 설계 실습으로 이어지는 이유가됨.)

(Q07의 핵심인 "게임이 여러 하드웨어 블록으로 어떻게 구성되는가

cat invaders.v 입력하여 전체구조를 봄

실제로 invaders.v에서는 ctrl → 게임 전체 제어, paddle → 플레이어 패들 움직임,

alien → 외계인 + 외계인 포탄, lfsr_4bit → 외계인 포탄 발사 위치 결정에 사용

ball → 공의 움직임 + 충돌, 마지막 assign pixel → 각 블록의 화면 픽셀을 하나로 합침

그리고 중요한 점은 이 모든 블록이 프로그램처럼 하나씩 실행되는 게 아니라 하드웨어 회로로 동시에 동작한다는 것이다.

이게 우리가 앞에서 얘기했던 소프트웨어의 순차 실행 ↔ Verilog의 병렬적인 하드웨어 동작과 바로 연결이된다.)

(이후 추가적인게임의 시작 진행 게임오버/클리어를 어떻게 제어하는지 보기위해 cat ctrl.v를 재차 입력함.그러자 게임의 진행 순서를 관리하는 중앙 제어 회로 파일이 나왔고 그안에는 하드웨어설계에서 정말 중요한 FSM(Finite State Machine, 유한상태기계)이 등장. 즉 현재 상태에 따라 다음에 할 하드웨어 동작을 결정하는 구조)

(추가로 외계인이 쏘는 포탄의 위치가 바뀌는걸 알아보았는데, 4비트값을 이용해 마치 랜덤숫자를 만드는거처럼보이는 LFSR(하드웨어에서의 간단한 의사난수 패턴을 만드는회로)을 활용하여 진행한거였음.)

과제 1: 기존 코드와 AI가 개선한 코드를 diff로 비교하고, AI가 만든 코드를 이해할 수 있는지 생각하기

: 기존 코드와 AI가 개선한 코드를 diff 명령을 이용하여 직접 비교하였다. 단순히 AI가 작성한 코드를 사용하는 것이 아니라, 기존 코드와 비교하면서 어떤 부분이 추가되거나 변경되었는지 확인

키보드 입력을 담당하는 sc_invaders_TB.cpp를 기존 코드와 개선된 코드로 비교하였다. 또한 GLCD 출력을 담당하는 sc_glcd128x64_TLM.cpp도 기존 코드와 개선된 코드를 비교하였다.

diff 결과를 확인하면서 AI가 작성한 코드에는 기존 기능을 유지하면서 키보드 입력이나 그래픽 출력과 관련된 기능을 추가하거나 수정한 부분이 있다는 것을 확인

특히 AI가 생성한 코드를 그대로 사용하는 것과 코드를 직접 이해하는 것은 차이가 있다는 점을 알 수 있었다.

이번 과제를 통해 AI를 이용하면 코드를 빠르게 작성하거나 수정할 수 있지만, 생성된 코드를 그대로 사용하는 것보다는 기존 코드와 비교하고 변경된 부분의 동작 원리를 이해하는 과정이 중요하다는 것을 알게 되었다.

과제 2: AI가 만들어준 SystemC용 Makefile로 실제 빌드해보기

: 과제2 오류나는 코드인 make -f Makefile.mk 입력하였음.

입력하면 나오는 Cannot find file containing module: 'space_invaders_engine_glcd.v'가 핵심이다. 즉 Makefile이 space_invaders_engine_glcd.v를 현재 Q09e 폴더에서 찾으려고 하는데, 그 파일이 없어서 실패한 것.

따라서 Q09b 폴더에 Verilog파일이 있는 것을 확인후 파일경로를 해당경로로 수정하였다.

그러나 추가적으로 오류가발생했다. SystemC와 Verilator 간의 신호 타입 차이로 인해 `sc\_uint` 관련컴파일오류가 발생하였다. 이를 해결하기위해 Verilator옵션에 `--pins-sc-uint`를 추가하였다. 또한 Q09b에 있는 SystemC 소스 및 헤더 파일을 Q09e에서 참조하도록 경로를 수정하였다.

수정 후 `obj\_dir`을 삭제하고 다시 빌드한 결과 컴파일 및 링크가 정상적으로 완료되었으며,

생성된 실행 파일을 실행하여 '하드웨어 리셋해제완료' 메시지를 확인하였다. 따라서 Verilog 하드웨어를 SystemC 환경으로 변환하고 실행하는 과정을 성공적으로 완료하였다.

```
=====
[SUCCESS] 아케이드 빌드가 성공적으로 완수되었습니다!
-> 실행 바이너리 실행 지점 명령: './obj_dir/Vspace_invaders_engine'
=====

./obj_dir/Vspace_invaders_engine_glcd

SystemC 3.0.2-Accellera --- Sep 15 2026 01:55:13
Copyright (c) 1996-2025 by all Contributors,
ALL RIGHTS RESERVED
[SystemC TB] 하드웨어 리셋 해제 완료. 시뮬레이터 조작을 시작합니다.
```

<이해를 돕기위한 이미지이다.>

이번 실습을 통해 Verilog HDL이 단순한 프로그래밍 언어가 아니라 실제 하드웨어의 구조와 동작을 표현하는 기술 언어라는 점을 이해할 수 있었다.

또한 실습 중 생긴 다양한 문제들을 직접해결하며 단순히 코드를 작성하는 능력뿐만 아니라 빌드환경구성 및 오류 분석 해결하는 능력도 중요하다는 것을 경험하였다.